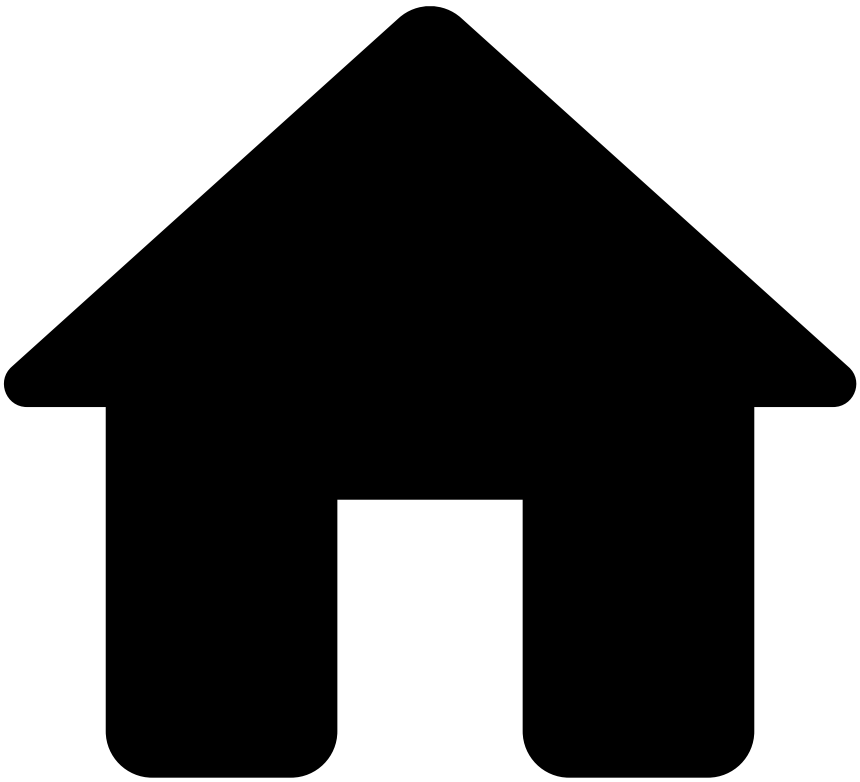


On this page

App Collaboration Loop

Was this page helpful?Â 💬

To build a Fraud Prevention solution, Rules and Data Science resources such as Plans and Models have to be developed, tested, and then deployed into Production to score transactions in real-time. These stages are often performed in different environments to ensure that the work in Development does not affect the work in Production. The stages are also performed by different teams at Feedzai and from Customers, including Data Scientists, Engineering Teams, and Fraud Analysts. The concept of collaborating on a Fraud Prevention solution using Pulse Applications (**Pulse apps**) is known as **App Collaboration**.

Similarly to the Data Science Loop, **App Collaboration** is not a one-time process. It requires a constant integration between the work developed in offline environments with the work in Production. This concept is described in this guide as **App Collaboration Loop**.

This guide covers the following two key ideas:

- How to structure an application in different environments.
- Workflow between environments.

Introduction to App Collaboration Loopâ□□

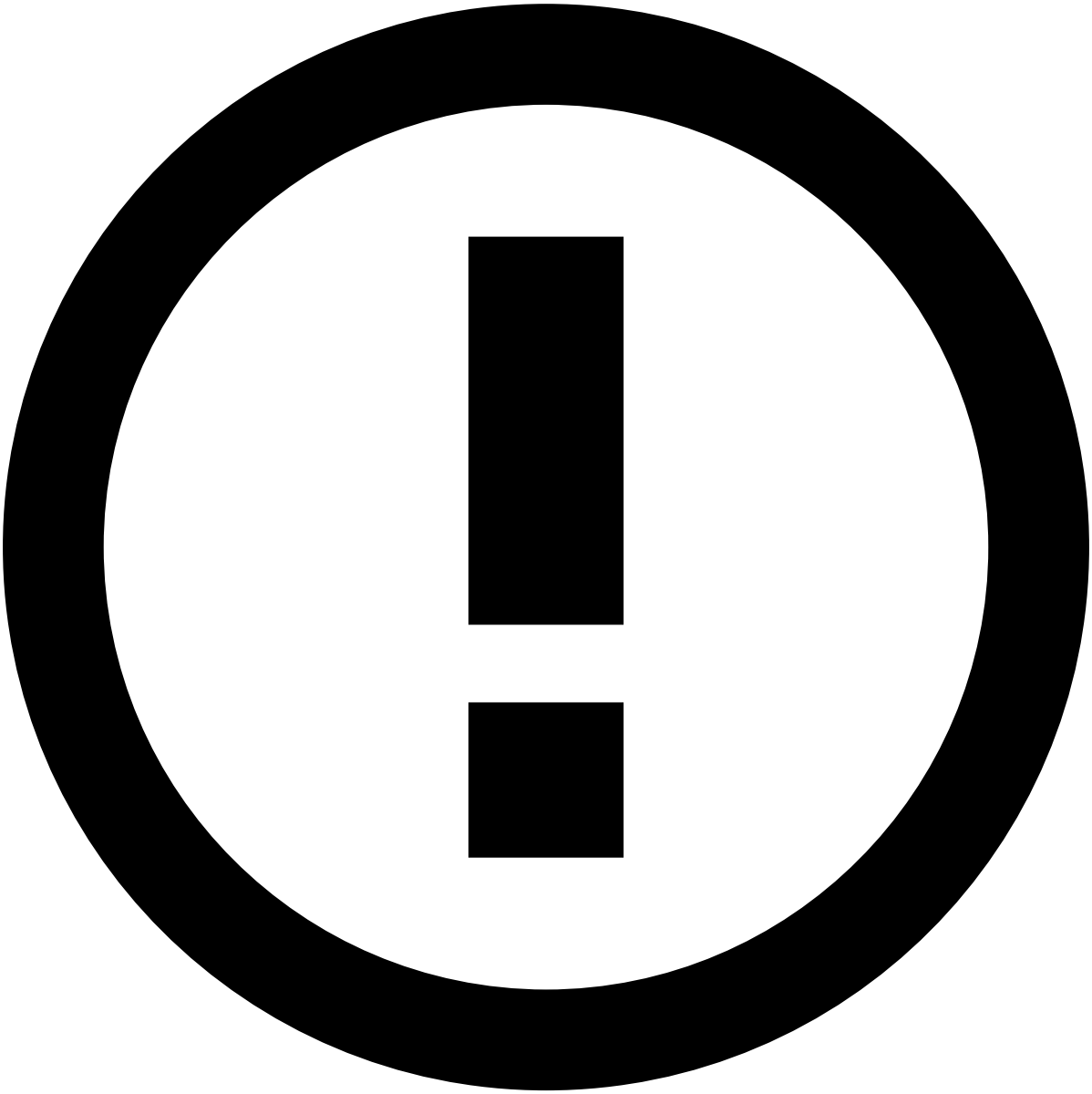
App Collaboration Loop sets a working methodology so that different teams can collaborate together by developing various resources simultaneously and then bring the latest improvements faster and reliably to Production. **App Collaboration Loop** is recommended whenever you want to update Pulse resources (e.g. **Models**, **Plans**, Rules, Lists).

This working methodology is performed by different teams and requires different environments as follows:

- Offline environments:
  - **Data Science environment (DS):** Used by Data Science teams to develop tasks included in the **Data Science Loop** (e.g. improve **Models**).
  - **Development environment (DEV):** Used by Engineering teams to test more complex **Rules**, add schema fields, as well as to integrate Pulse resources developed in DS and in Production with the resources evolved in DEV.
- Runtime environment:
  - **Production environment (PROD):** Used by Analysts to process transactions in real-time, and to write Self-service Rules. PROD is set up and then continuously updated by the Engineering team (e.g. update **Pulse app** or software versions).

Block ownershipâ□□

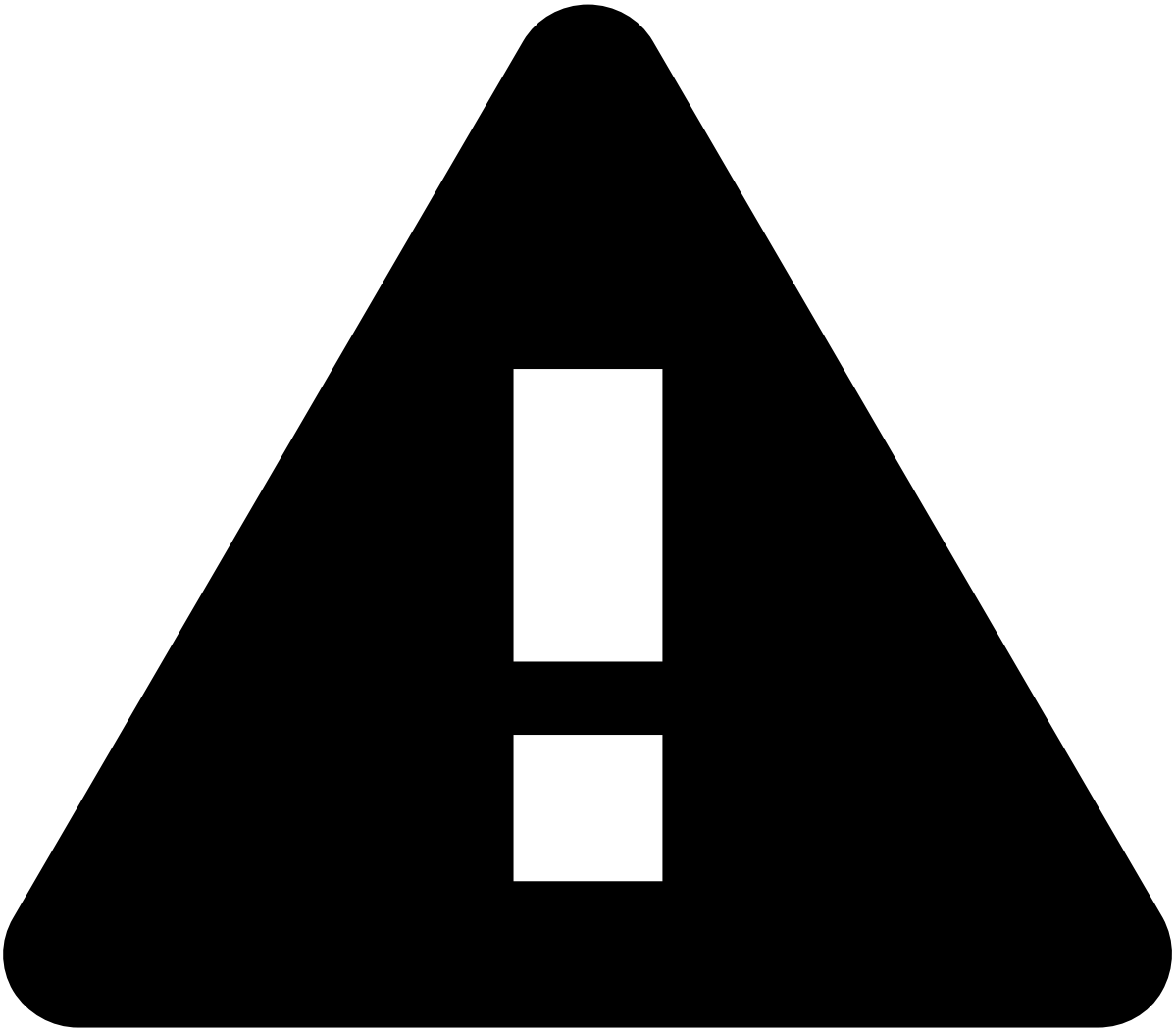
If you are getting started with the App Collaboration approach, this section will help you understand how to set up your environment using **Block ownership** to develop Pulse resources.



App Collaboration approach

If you are already familiarized the **Block ownership** concept, you can skip this section and visit the section Workflow between environments to learn how to evolve a **Pulse app** from DEV/DS to PROD.

To evolve an application in a collaborative way, App Collaboration uses the concept of Block ownership. With this concept, each Pulse resource is evolved as a block by a single team within a single environment. As an example, a machine learning Model is evolved only by Data Scientists within the DS environment. The image below illustrates a typical example of three environments (DEV, DS and PROD) using Block ownership:



caution

The block ownership illustrated above assumes the following:

1. Between each deployment, each block evolves in a single environment (i.e. in DEV, DS, or PROD).
2. For each environment, all the resources are evolved using a single Pulse cluster.

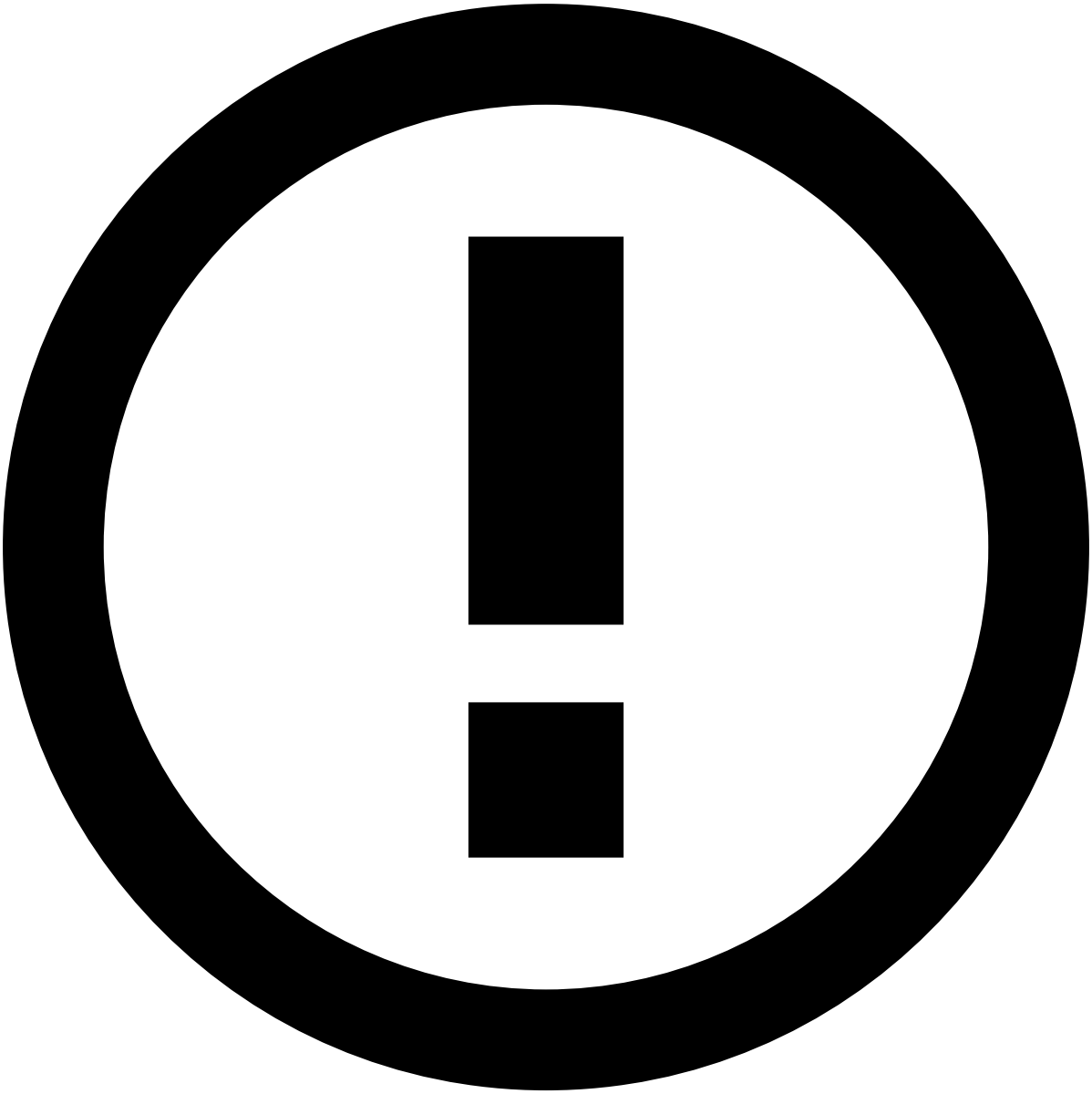
Some resources are often developed by various teams (i.e. owners), for example, Rules Projects. Using the concept of resource ownership, each Rules Project should be owned by one team and should be only modified (e.g. adding/ removing/editing Rules) by the team that owns each Rules Project. One possible solution is to divide the Rules Projects as follows:

- ***Eng. Rules:*** Represents Rules Projects with higher complexity, which are only evolved by Engineering teams in the DEV.
- ***DS Rules:*** Represents Rules Projects that are closely related to Data Science, which are only evolved by Data Scientists in DS.
- **Self-service Rules:** represents Rules Projects containing simple Rules which are only written by Analysts in PROD, using Case Manager.

See the page **How to organize your work with blocks** to learn to set a **Pulse app** with this structure.

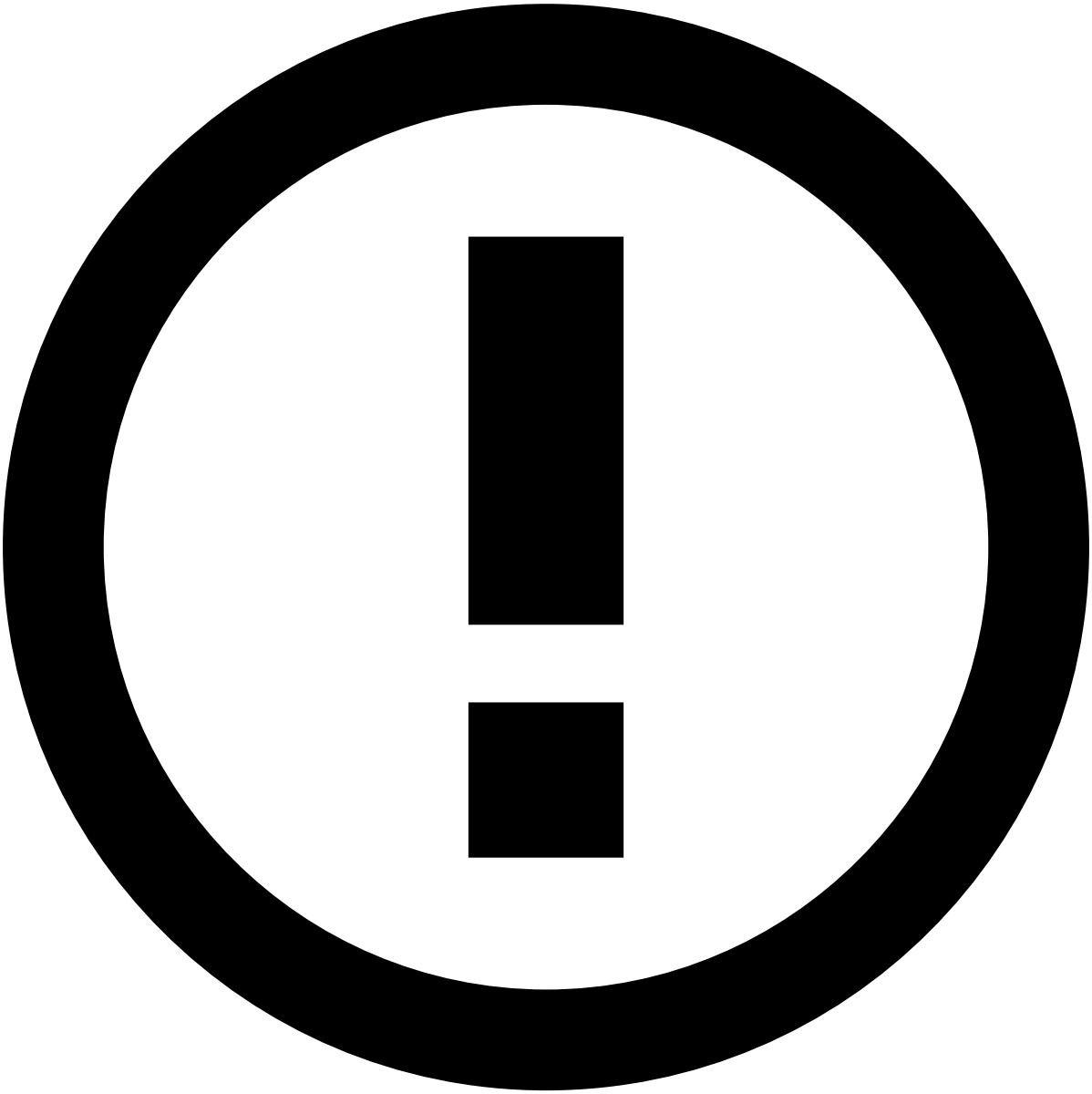
Workflow between environmentsâ

The figure below illustrates the procedure to bring new improvements to PROD after the initial deployment:



When to use this approach

You should follow the procedure below to deploy any resource type (e.g. Models, Plans, **Rules**, **Lists**). This procedure allows Fraud Analysts to use PROD to score transactions and evolve SSR, while Data Scientists and Engineers evolve resources in DS and DEV.



App Collaboration approach

The workflow explained in this section is specially designed to be adopted after the initial deployment, i.e. when you have already a live solution, to continuously improve the project with subsequent deployments.

The numbers shown in the diagram represent the following steps:

- **Step 1:** Export the resources that are being developed in DS and import them in DEV. See the Section Migrating Data Science work to DEV for more information about this step.
- **Step 2:** Export SSRs from PROD to DEV. This is required because SSRs might have changed while you evolved your applications in DEV and DS. Therefore, you need to ensure that the SSRs being written in PROD are migrated to DEV to test the full solution before the new deployment. See the Section Migrating SSRs from PROD to DEV for more information about this step.

After having your work fully developed and tested, you can bring the latest version of your application to PROD. One option is to perform a blue-green deployment.

The steps above are broken down into key topics in the next subsections.

Step 1: Migrating Data Science work to DEVâ□□

Migrating the resources to DEV requires loading them in the Data Science tab and applying them in the runtime Workflow. Check the page How to organize your work with blocks to understand how to map resources between Data Science tab and runtime Workflow.

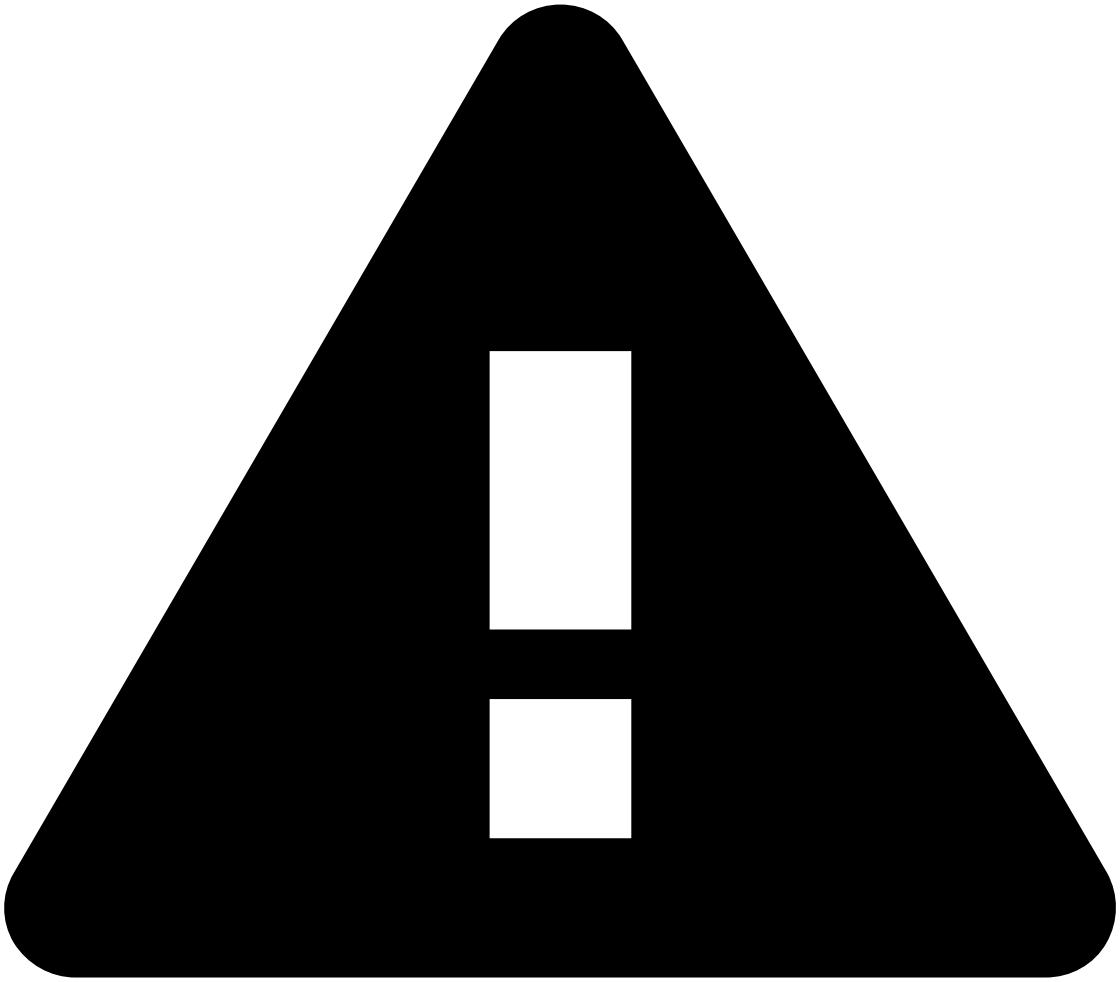
The following pages will help you to accomplish this stage:

- **Exporting** and **Importing Lists**
  Allows you to migrate Lists to DEV. By default, when you run the export and import wizards, all the Lists are migrated automatically (i.e. this only include List metadata). All the other resources (i.e. Plans, Models and Rules Projects) can be optionally migrated using the selection options provided on the wizards.

- **Exporting** and **Importing Plans**
  Allows you to migrate the work related to the Feature Engineering stage of the Data Science loop. This includes the configurations that add fields to your dataset schema once the Plan is executed in the new environment (e.g. to add derived features, historical profiles). Optionally, you can choose to migrate Plan executions. Note that, every time that Data Scientists works on a Plan that has impact on the Data Source, the Model has to be re-trained. In this case, you also need to export and import the Model.

- **Exporting** and **Importing Models**
  Allows you to migrate the Model configurations, and optionally, you can include the binary of the Model.

- **Exporting** and **Importing Rules Projects**
  Allows you to migrate the block of Rules developed by the Data Scientists in DS. By exporting and importing Rules Projects you are migrating the entire Rules Project with its Rules, and optionally, you can include a list of Rules Snapshots.

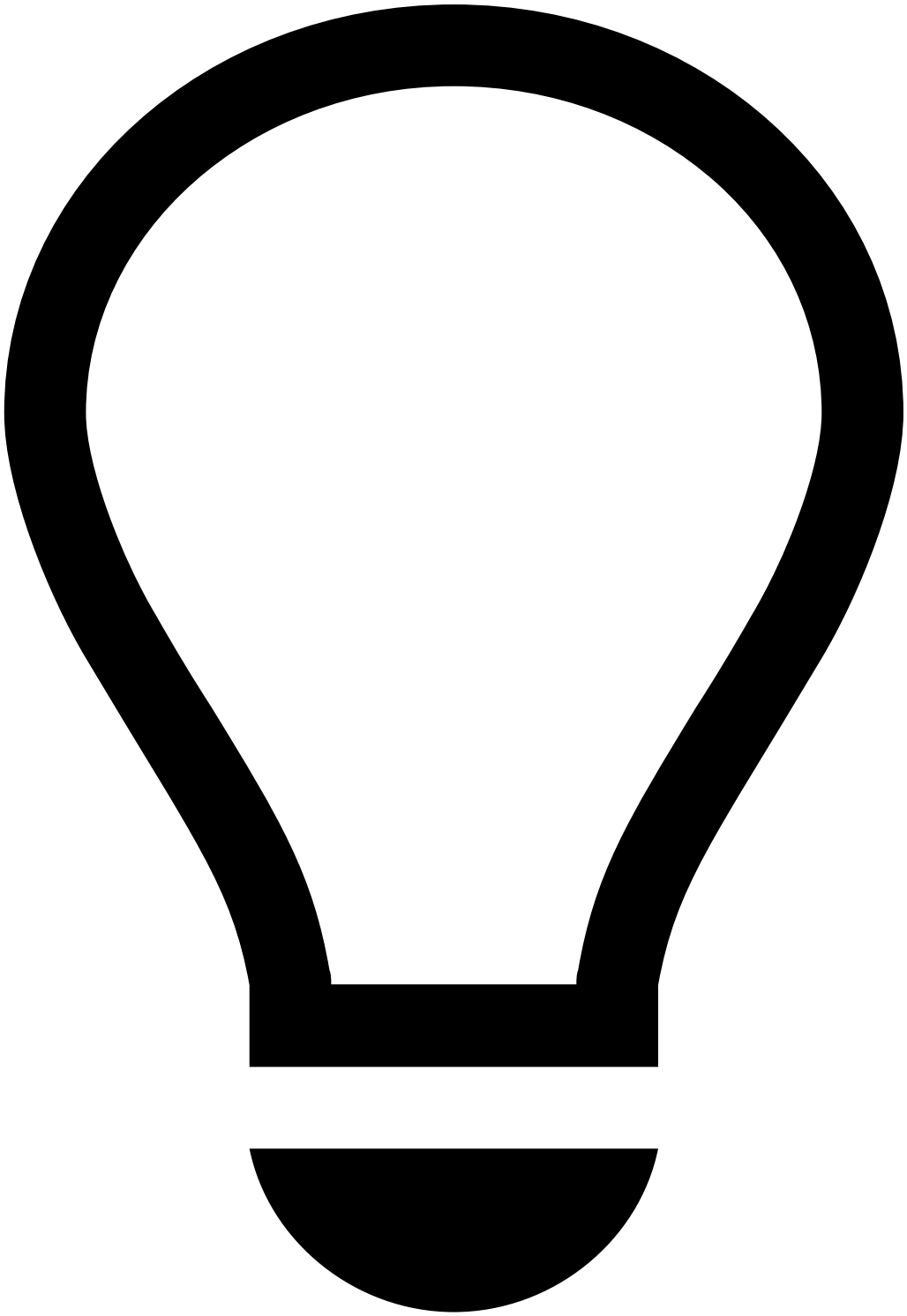


Block ownership - Rules Projects

Feedzai strongly recommends to evolve Rules Project using the concept of Block ownership to get the most out of the App Collaboration Loop with multiple teams working on the project. If multiple teams make changes simultaneously in the same Rules Projects using different environments (i.e. not adopting the resource ownership), you will have to merge changes manually by opening and editing the JSON file obtained by the Export Resource tool.

In addition to the links above, consider the following typical scenario:

- An engineer is working on a Plan to add derived features to a data schema in the DEV environment. At the same time, a Data Scientist is working on a Rules Project that is using the old schema. The Import resource option allows you to choose the new schema when importing the Rules Project from DS to DEV, assuming that the new schema is already configured in that Rules Project in the DEV environment.



tip

Check the section Schema Conflicts in the page Importing resources to learn how to perform this operation.

Step 2: Migrating SSRs from PROD to DEVâ□□

During the DS and DEV work, Fraud Analysts might evolve the blocks of SSRs in PROD. Completing this stage ensures that you integrate the most recent changes in PROD with the new improvements in DEV. This stage should be done before bringing the new app version to PROD to test the full solution before the new deployment. The easiest way to accomplish this stage is by exporting and importing SSRs from PROD to DEV, respectively.

The following pages will help you to accomplish this stage:

- **Exporting** and **Importing SSR**
  In PROD, you can use the Export Resources wizard to select the Rules Projects related to SSRs and export as a zip file. This file can be then imported in DEV using the Import Resources wizard.

Learn Moreâ□□

The following pages allow you to complete different tasks related to the App Collaboration Loop:

- How to organize your work with blocks
- How to version and store
- How to merge
- How to code review
- Errors in App Collaboration

Additionally, learn more about the concept of App Collaboration by checking the User Guide, including the following key pages:

- Exporting Resources
- Importing Resources